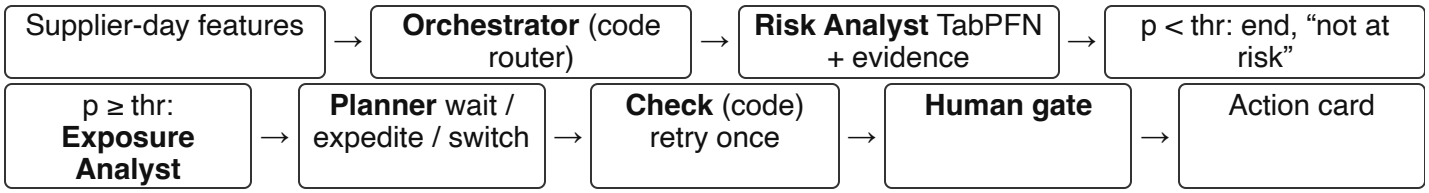
Supplier disruption detection and replenishment agents

Lakshya Sharma · Track A (Agentic / Multi-Agent AI) · Scenario S1 (Supply Chain Risk) ·
github.com/lakshya082001/FMs-AgenticAI

1. Problem & Domain

Problem: detect supplier disruptions early from operational data, then recommend replenishment actions (wait, expedite, switch supplier) before stock-outs, with a human sign-off on risky plans. **Why Track A:** B has no language task needing weight adaptation; C would end at a risk score, but the value is the action and its review; D has no document corpus (structured data). Why agentic rather than one LLM or a classifier alone: the task needs tools, an independent check between generation and action, and a human gate, which a single prompt or a bare probability cannot give.



Every node returns to the orchestrator, which logs its decision and reason. Gate fires on borderline p, cost over 10,000, or a failed check.

2. Approach & Algorithm Decisions (choice → alternative rejected)

- **Why a model at all, not agents alone?** Spotting a disruption needs relations between many features (late rates, in-transit delays, volume), which an LLM cannot compute reliably. Those relations can be built as a graph (see this Neo4j agentic design; I could not open the full article, so I rely on its description of agents querying a graph database) or learned by a model. A graph needs more exhaustive relational data and build time, so we use a structured-data foundation model: **TabPFN** gives probabilistic predictions on small datasets with no hyperparameter tuning ("fit" just stores the context), and one agent holds that prediction. *Rejected:* graph DB (data and time); XGBoost (a fair baseline I did **not** run, so I claim no win over it); sequence models such as LSTM/Chronos (3.6K rows, short episodes, trailing-window features already summarise history).
- **Closest prior work** arXiv 2601.09680: 7 CrewAI/GPT-4o agents, news-triggered, Neo4j multi-tier graph, rule-weighted risk score, 30 expert scenarios (F1 0.90–0.99), no confidence scores and no separate verification step described. *We differ:* structured-data trigger, calibrated probability, deterministic verification with retry, an executable human gate, traced failure cases. They are stronger on multi-tier graph reasoning and expert-labelled evaluation.
- **Data: real DataCo orders + simulated disruptions.** The raw late flag has no temporal signal (lane-level weekly autocorrelation –0.02 to –0.06), so detection on raw labels would be vacuous. *Rejected:* fully synthetic data with planted rules (circular). We overlay seeded disruption episodes (delay days, not just flipped labels) on real order flow; severity was never tuned for results.
- **Features:** trailing 3–90 day late rates, deviations, and the share of in-transit orders past their promised date (observable before delivery completes). *Rejected:* completed-delivery rates alone (they lag the true state by shipping time; first attempt AUC 0.62 val / 0.66 test).
- **Orchestration: LangGraph** (explicit state, conditional edges, retry loop, native interrupt for the human gate). *Rejected:* CrewAI / AutoGen, as sequential or chat-style flows fit a retry-and-gate loop less cleanly (my judgement, not benchmarked).
- **Decomposition:** 3 LLM agents + a code orchestrator + a code check, mapped to detect → exposure → action. *Rejected:* one LLM with tools (no independent verification, single failure point); an LLM critic (a deterministic number check is cheaper and cannot hallucinate); merging Exposure into Planner (hides an inspectable intermediate result).
- **Planner:** LLM picks among tool-computed options (it never computes numbers). *Rejected:* pure argmin/OR-Tools, which would be optimal, but the LLM adds rationale; the check bounds it to within 10% of the cheapest option.

- **Silent failures found and handled:** the LLM stated “0.27 is below 0.25” (alert status and hedge word now computed in code); it missed the cheapest option on 3 of 15 early decisions (2 distinct SKUs, one ~15K worse) (optimality check + retry); injected wrong numbers are caught by the number check; failures escalate to a human.

3. Results & Error Analysis

Target: supplier is inside a disruption episode that day. **11 features:** late rate over 3/7/14/30/90 days, 7-day volume, two deviations, days since last alert, in-transit overdue count and share. 8 suppliers; train 3,656 rows (Apr 2015–Jun 2016), val 1,472, test 2,184 (Jan–Sep 2017), temporal split.

Component	One-liner
Risk Analyst (LLM + TabPFN)	Validates the row, runs the pickled TabPFN, writes a hedged evidence summary.
Exposure Analyst (LLM)	Finds the supplier's SKUs whose days of cover fall short of disruption-adjusted lead time.
Planner (LLM)	Picks wait, expedite or switch per SKU from a tool-computed cost table.
Orchestrator, Check (code)	Route and log every step; verify numbers and cost-optimality, retry once, escalate.

	AUC	Avg prec.	Brier	ECE	Base rate	Best single feature (od_share) AUC
Val	0.659	0.298	0.121	0.071	0.145	0.689
Test	0.652	0.402	0.172	0.091	0.234	0.664

At threshold 0.25 (best F1 on val), 27 of 35 test episodes are detected, median delay 5 days, false-alarm rate 17.4%. **TabPFN does not beat the one-feature baseline**; gains came from the in-transit overdue signal, not the model. Single seed and 35 episodes: ~0.01 AUC differences are noise. Eight committed traces cover success, borderline, retry after a failed check, false alarm, invalid input and fault injection; a run takes 40–46 s (about 30 s is single-row TabPFN inference on CPU), 1–4 LLM calls, ~2–3.9K tokens. Latency is measured per run only and dollar cost is not computed.

Where it fails: long-range temporal dependence (windows stop at 90 days; episodes are memoryless, so seasonality and recurring supplier fragility are untested); train-to-test shift (15% → 23% disrupted) hurts calibration; detection lags episode start by ~5 days because deliveries complete late; false alarms trigger real plans (a supplier at p=0.71 that was not disrupted still got a plan); alternates are assumed undisrupted and exposure is single-tier.

4. Production & Limitations

Production: export the fitted TabPFN weights and serve them (or the hosted API) with batch daily scoring on a GPU and cached scores, since single-row CPU inference takes ~30 s; run the agent graph as a service with a persistent LangGraph checkpointer so human approvals survive restarts; monitor alert rate, feature drift and calibration once labels mature. If multi-tier reasoning is needed, add a graph DB (e.g. Neo4j) holding the supplier network. **Next:** a what-if agent that sweeps features to show what would flip an alert, and per-node latency/cost tracing. **Limitations:** disruptions are simulated, so results measure detection of simulated events, not real-world accuracy; TabPFN weights used (v3.5) carry a non-commercial license.